IT Infrastructure Task 4 NFS Accounts and Intrusion Detection

This report documents NFS sharing, departmental access controls, and Snort rule configuration and testing.

NFS

Task 1: Install NFS on Ubuntu Server

Install nfs kernel server to run nfs for the server

```
sang@22080031:~$ sudo apt install nfs-kernel-server -y
[sudo] password for sang:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nfs-kernel-server is already the newest version (1:2.6.1-1ubuntu1.2).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

The server installs nfs-kernel-server, which provides the NFS service used to export directories. The package manager reports that it is already installed in this VM.

Check server's IP address which is 192.168.56.118

```
sang@22080031:~$ sudo ifconfig
[sudo] password for sang:
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.56.118  netmask 255.255.255.0  broadcast 192.168.56
        inet6 fe80::a00:27ff:fe50:98a6  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:50:98:a6  txqueuelen 1000  (Ethernet)
        RX packets 12  bytes 2363 (2.3 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 68  bytes 9288 (9.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.12  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::a00:27ff:fe78:5a0f  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:78:5a:0f  txqueuelen 1000  (Ethernet)
        RX packets 34  bytes 7565 (7.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 102  bytes 11874 (11.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
```

The interface listing shows the server at 192.168.56.118 on the lab network. The client uses this address as the source of both NFS mounts later in the exercise.

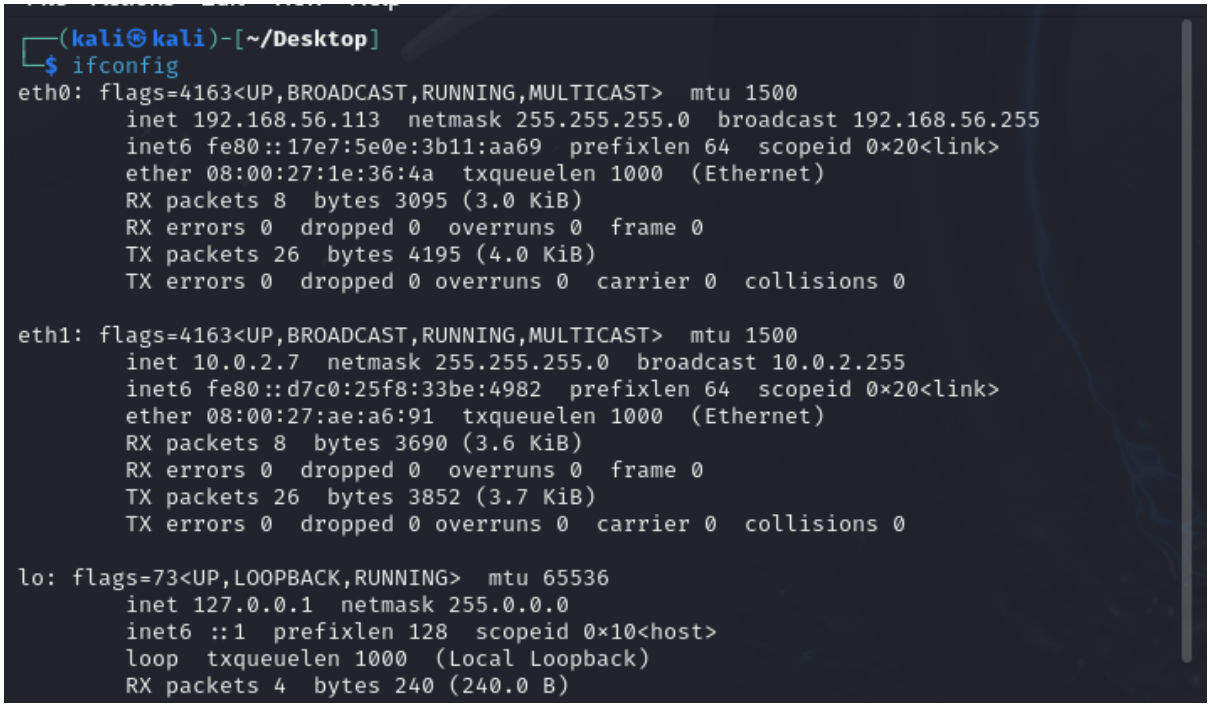
Enable and start the service

```
sang@22080031:~$ sudo systemctl enable nfs-server
sang@22080031:~$ sudo systemctl start nfs-server
```

The service commands enable NFS at boot and start it for the current session. The server is ready to publish shares after /etc/exports is configured.

Task 2: Install NFS Client on the Linux Client

Check client's IP address which is 192.168.56.113



The client screenshot shows 192.168.56.113 on the same /24 network as the NFS server. The pictured client is Kali Linux, despite the section heading calling it an Ubuntu client.

Update the system

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo apt update
[sudo] password for kali:
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Get:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [18.2 kB]
Get:4 https://packages.microsoft.com/repos/code stable/main arm64 Packages [18.4 kB]
Get:5 https://packages.microsoft.com/repos/code stable/main armhf Packages [18.5 kB]
Get:3 http://kali.download/kali kali-rolling InRelease [41.5 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 Packages [20.8 MB]
Get:7 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [50.6 MB]
Get:8 http://kali.download/kali kali-rolling/contrib amd64 Packages [119 kB]
Get:9 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [326 kB]
Get:10 http://kali.download/kali kali-rolling/non-free amd64 Packages [201 kB]
Get:11 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [882 kB]
Get:12 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [10.6
kB]
Get:13 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb)
[24.3 kB]
Fetched 73.1 MB in 7s (10.3 MB/s)
```

The client refreshes its package lists before installing NFS utilities. This allows apt to use the repositories configured on the client system.

Install nfs common to run nfs on the client

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo apt install nfs-common -y
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
The following packages were automatically installed and are no longer required:
  cpp-13 fonts-noto-color-emoji libboost-iostreams1.74.0 libboost-thread1.74.0
  libnsl-dev libroc0.3 libsnapd-glib-2-1 libtirpc-dev
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  dmsetup ldap-utils libdevmapper1.02.1 libevent-core-2.1-7t64
  libevent-openssl-2.1-7t64 libevent-pthreads-2.1-7t64 libldap-common libldap2
  libnfsidmap1 libtirpc-common libtirpc-dev libtirpc3t64
Suggested packages:
  libsasl2-modules-gssapi-mit | libsasl2-modules-gssapi-heimdal open-iscsi watchdog
The following packages will be REMOVED:
  libevent-core-2.1-7 libevent-openssl-2.1-7 libevent-pthreads-2.1-7 libtirpc3
The following NEW packages will be installed:
  libevent-core-2.1-7t64 libevent-openssl-2.1-7t64 libevent-pthreads-2.1-7t64
  libldap2 libtirpc3t64
The following packages will be upgraded:
```

The nfs-common package supplies client-side NFS tools, including support for the mount commands used below. The screenshot shows the install being requested on the Kali client.

Task 3: Create Directories on Server

Make directory for mounting public for everyone can access, and private for limited client access

```
sang@22080031:~$ sudo mkdir -p /mnt/nfs/public
sang@22080031:~$ sudo mkdir -p /mnt/nfs/private
```

The server creates separate public and private directories under /mnt/nfs. These paths become the export roots named in /etc/exports.

```
sang@22080031:~$ sudo chmod 755 /mnt/nfs/public
sang@22080031:~$ sudo chmod 777 /mnt/nfs/private
```

The chmod commands set the public directory to 755 and the private directory to 777. Mode 777 gives every local user read, write, and execute permission, so the private name does not by itself make that directory private on the server.

Change permission for 2 directories according to the following:

Public:

	Read	Write	Execute
User	yes	yes	yes
Group	yes	no	yes
Other	yes	no	yes

Private:

	Read	Write	Execute
User	yes	yes	yes
Group	yes	yes	yes
Other	yes	yes	yes

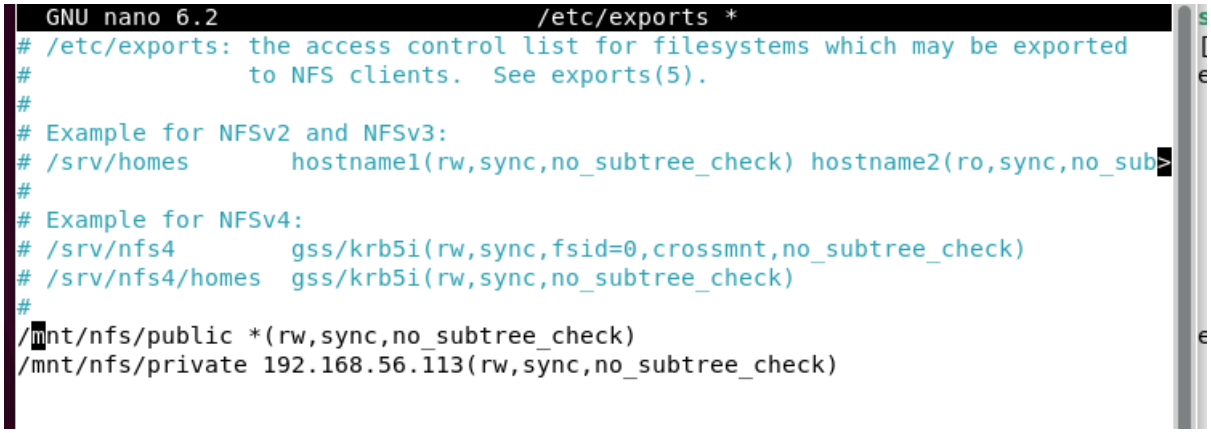
The permission table translates those modes into owner, group, and other access. Public is writable only by its owner; private is writable by everyone locally, and its network access relies on the export rule.

Task 4: Configure exports file on Server

Access the export file to define which directories on the server are shared with NFS clients and specify the access permissions for each one.

```
sang@22080031:~$ sudo nano /etc/exports
```

The server opens /etc/exports, the file that controls which paths NFS clients may mount. Each line combines a local directory, an allowed client set, and mount options.



The following table explain the configuration:

The public export uses * to allow any client that can reach the server, while the private export names only 192.168.56.113. Both entries allow read and write access and use sync and no_subtree_check.

Commands	Description
/mnt/nfs/public *(rw,sync,no_subtree_check)	• **/mnt/nfs/public**: This is the directory on the NFS server that is being shared. • ***: This means **all clients** (any IP) are allowed to access this share. • **Options**: • **rw**: Read and write access. • **sync**: Changes are committed to disk immediately (ensures data integrity, but slower). • **no_subtree_check**: Skips checking the parent directory's path when accessing subdirectories (improves performance and avoids issues if the directory structure changes).
/mnt/nfs/private 192.168.56.113(rw,sync,no_subtree_check)	• **/mnt/nfs/private**: Another directory being shared. • **192.168.56.113**: Only the client with this specific IP address is allowed access. • **Options**: • Same as above (rw, sync, no_subtree_check).

The export table explains that * is broad access and the single client address restricts who can mount the second share. Export restrictions do not replace local filesystem permissions or a network firewall.

allow export all directories listed in the above configuration, then restart the service.

```
sang@22080031:~$ sudo exportfs -a
sang@22080031:~$ sudo systemctl restart nfs-server
```

exportfs -a applies the listed exports, and the NFS service is restarted. These actions make the updated /etc/exports settings available to client mount requests.

Task 5: Mount NFS Directories on the Client Side

Stop the firewall service

```
sang@22080031:~$ sudo systemctl stop ufw
```

The server's UFW service is stopped before testing NFS. This removes a firewall obstacle in the lab, but it also removes the server's firewall protection; a deployed system should permit the required NFS traffic explicitly.

Create the public and private directory in the client

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo mkdir -p /mnt/nfs/public

┌──(kali㉿kali)-[~/Desktop]
└─$ sudo mkdir -p /mnt/nfs/private
```

The client creates matching local mount-point directories. These empty paths will display the server's files once the remote shares are mounted.

Now mount client's public and private with server's public and private respectively

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo mount -t nfs 192.168.56.118:/mnt/nfs/public /mnt/nfs/public

┌──(kali㉿kali)-[~/Desktop]
└─$ sudo mount -t nfs 192.168.56.118:/mnt/nfs/private /mnt/nfs/private
```

The two mount commands connect the client paths to the server's public and private exports at 192.168.56.118. A successful mount makes remote files appear under /mnt/nfs on the client.

Task 6: Create a File with HAN Number in Private Directory

writes `22080031` into `22080031.txt` inside the `/mnt/nfs/private/`

```
┌──(kali㉿kali)-[~/Desktop]
└─$ echo "22080031" | sudo tee /mnt/nfs/private/22080031.txt
22080031
```

The client writes the student number into a file inside the mounted private share. Because the path is mounted over NFS, the file is written to the server's exported directory.

Task 7: Access the File from the Server

Go to the server and verify by displaying the file in private folder and displaying the content of that file

```
sang@22080031:~$ ls -l /mnt/nfs/private/
total 4
-rw-r--r-- 1 nobody nogroup 9 Apr  8 18:57 22080031.txt
sang@22080031:~$ cat /mnt/nfs/private/22080031.txt
22080031
```

The server lists the new file and reads its contents, confirming that the client write reached the NFS export. The displayed nobody/nogroup ownership is consistent with an unmapped or squashed remote identity in this lab.

Account management (AM)

Task 1: Users and groups creations

Create 8 users from 1 to 8

```
sang@22080031:~$ sudo useradd -m user1
sang@22080031:~$ sudo useradd -m user2
sang@22080031:~$ sudo useradd -m user3
sang@22080031:~$ sudo useradd -m user4
sang@22080031:~$ sudo useradd -m user5
sang@22080031:~$ sudo useradd -m user6
sang@22080031:~$ sudo useradd -m user7
sang@22080031:~$ sudo useradd -m user8
```

Eight local user accounts are created with home directories. These accounts provide test identities for the departmental group and permission exercise.

Set password for each user from 1 to 8

```
sang@22080031:~$ sudo passwd user1
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
sang@22080031:~$ sudo passwd user2
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
Sorry, passwords do not match.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
```

The password commands assign credentials to the new accounts. The screenshot includes short-password warnings, so successful account creation should not be taken as proof of strong password policy.

Print the /etc/passwd file to verify the number of user

```
sang@22080031:~$ cat /etc/passwd
```

The first /etc/passwd capture shows the newly added user entries. This file records account names and IDs, but it does not reveal their passwords.

```
/nologin
hplip:x:127:7:HPLIP system user,,,:/run/hplip:/bin/false
_rpc:x:128:65534::/run/rpcbind:/usr/sbin/nologin
statd:x:129:65534::/var/lib/nfs:/usr/sbin/nologin
user1:x:1001:1001::/home/user1:/bin/sh
user2:x:1002:1002::/home/user2:/bin/sh
user3:x:1003:1003::/home/user3:/bin/sh
user4:x:1004:1004::/home/user4:/bin/sh
user5:x:1005:1005::/home/user5:/bin/sh
user6:x:1006:1006::/home/user6:/bin/sh
user7:x:1007:1007::/home/user7:/bin/sh
user8:x:1008:1008::/home/user8:/bin/sh
```

The continuation of /etc/passwd shows user1 through user8. Together with the first capture, it verifies that all eight test accounts exist.

There are exactly 8 users that we print above

Add 4 groups for 4 departments: IT, HR, Marketing and Finance

```
sang@22080031:~$ sudo groupadd IT
sang@22080031:~$ sudo groupadd HR
sang@22080031:~$ sudo groupadd Marketing
sang@22080031:~$ sudo groupadd Finance
```

Four groups are created for IT, HR, Marketing, and Finance. Groups provide a common identity for setting directory access by department.

```
sang@22080031:~$ sudo usermod -aG IT user1
sang@22080031:~$ sudo usermod -aG HR user2
sang@22080031:~$ sudo usermod -aG Marketing user3
sang@22080031:~$ sudo usermod -aG Marketing user4
sang@22080031:~$ sudo usermod -aG Marketing user5
sang@22080031:~$ sudo usermod -aG Finance user6
sang@22080031:~$ sudo usermod -aG Finance user7
sang@22080031:~$ sudo usermod -aG Finance user8
```

The usermod commands add each account to its assigned department group. The distribution is one IT user, one HR user, three Marketing users, and three Finance users.

Append user to the correct group according to the below table:

Department Group	User
IT	user1
HR	user2
Marketing	user3
	user4
	user5
Finance	user6
	user7
	user8

The accompanying table summarizes the intended user-to-group assignment. It is the reference used to check the later getent output.

Task 2: Directories creations

Make directory for each group:

```
sang@22080031:~$ sudo mkdir /Dit
sang@22080031:~$ sudo mkdir /Dhr
sang@22080031:~$ sudo mkdir /Dmar
sang@22080031:~$ sudo mkdir /Dfin
```

Four top-level directories are created, one per department. Each path will receive group ownership and access rules matching the assignment table.

Change the **group ownership** of the directory. Each group will be the owner of its directory

```
sang@22080031:~$ sudo chown :IT /Dit
sang@22080031:~$ sudo chown :HR /Dhr
sang@22080031:~$ sudo chown :Marketing /Dmar
sang@22080031:~$ sudo chown :Finance /Dfin
```

The chown commands set each directory's group owner to its department. Group ownership lets a single mode or ACL entry apply to all members of that department.

Then set full read, write and execute for user and group user. But other don't have any permission

```
sang@22080031:~$ sudo chmod 770 /Dit
sang@22080031:~$ sudo chmod 770 /Dhr
sang@22080031:~$ sudo chmod 770 /Dmar
sang@22080031:~$ sudo chmod 770 /Dfin
```

The chmod 770 commands give the owner and owning group full access and deny access to others. The later ACLs add selected cross-department exceptions to this base rule.

Task 3: Users and groups permissions

```
sang@22080031:~$ sudo setfacl -m g:IT:rwx /Dit /Dhr /Dmar /Dfin
sang@22080031:~$ sudo setfacl -m g:HR:rwx /Dhr /Dmar /Dfin
sang@22080031:~$ sudo setfacl -m g:HR:--- /Dit
sang@22080031:~$ sudo setfacl -m g:Marketing:--- /Dit /Dhr /Dfin
sang@22080031:~$ sudo setfacl -m g:Finance:--- /Dit /Dhr /Dmar
```

The setfacl commands grant IT access to every department and HR access to HR, Marketing, and Finance, while denying Marketing and Finance access to other departments. ACLs add these named-group rules on top of the standard owner/group/other bits.

The command sets Access Control Lists. `-m` modifies permissions, and `g:GROUP` targets a group for the specified directory. Therefore, the command set the access privileges like in the following:

Group	Directory access
IT	All
HR	HR, Marketing, Finance
Marketing	Marketing
Finance	Finance

The access matrix summarizes the intended result: IT can enter every directory, HR can enter three, and Marketing and Finance are limited to their own. It should be read alongside the actual ACL settings, because a matrix alone does not enforce permissions.

```
sang@22080031:~$ sudo visudo
```

visudo is opened to edit sudo policy safely. This is separate from filesystem access: a user may be able to enter a directory without being allowed to run administrative commands.

Edit visudo, which controls **sudo privileges** for users and groups, then edit as the following configuration:

```
%HR ALL=(ALL) ALL
%Marketing ALL=(ALL) NOPASSWD: /bin/false
%Finance ALL=(ALL) NOPASSWD: /bin/false
```

The sudoers screenshot gives HR broad sudo rights and adds narrow /bin/false entries for Marketing and Finance. Those entries do not grant useful administrative actions, but they should not be described as a general deny rule if other sudoers entries also apply.

Commands	Explainations
%HR ALL=(ALL) ALL	• Members of the **HR group** can run **any command** on **any host** as **any user**, with **sudo access**. • They **will be prompted for a password**.
%Marketing ALL=(ALL) NOPASSWD: /bin/false	• Members of **Marketing** can run only **/bin/false** as sudo, **without a password**. • /bin/false does nothing and immediately exits — this effectively **blocks sudo access** for this group.
%Finance ALL=(ALL) NOPASSWD: /bin/false	Same as above, **Finance** group has no meaningful sudo permissions.

The explanation table interprets the sudoers entries. HR can run commands through sudo after authentication, while the other shown entries permit only the no-op /bin/false command.

Now verify our configuration by the following ways

```
sang@22080031:~$ getent group IT
IT:x:1009:user1
sang@22080031:~$ getent group HR
HR:x:1010:user2
sang@22080031:~$ getent group Marketing
Marketing:x:1011:user3,user4,user5
sang@22080031:~$ getent group Finance
Finance:x:1012:user6,user7,user8
```

getent group lists the current membership of all four department groups. The output matches the user assignments made earlier in the exercise.

Use `getent` check all the user in existing groups

Then check the access permission of each directory

```
sang@22080031:~$ ls -ld /Dit /Dhr /Dmar /Dfin
drwxrwx---+ 2 root Finance   4096 Apr  1 06:33 /Dfin
drwxrwx---+ 2 root HR        4096 Apr  1 06:33 /Dhr
drwxrwx---+ 2 root IT        4096 Apr  1 06:32 /Dit
drwxrwx---+ 2 root Marketing 4096 Apr  1 06:33 /Dmar
```

The ls -ld output shows each directory's group owner and a trailing + on its permission string. The + indicates that extended ACL entries are present in addition to the basic 770 mode.

Finally, switch to a user in the list and try to access every directory to check the privilege

```
sang@22080031:~$ su - user3
Password:
$ cd /Dit
-sh: 1: cd: can't cd to /Dit
$ cd /Dmar
$ pwd
/Dmar
$
```

The user3 test fails to enter /Dit but succeeds in /Dmar. Because user3 belongs to Marketing, this is a concrete check of the intended access boundary.

Intrusion Detection

Task 1: Install Snort on Ubuntu Server

Install snort for intrusion detection system

```
sang@22080031:~$ sudo apt install snort -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libdaq2 libdumbnet1 libestr0 libfastjson4 libluajit-5.1-2 libluajit-5.1-common
  libnetfilter-queue1 logrotate oinkmaster rsyslog snort-common
  snort-common-libraries snort-rules-default
Suggested packages:
  bsd-mailx | mailx rsyslog-mysql | rsyslog-pgsql rsyslog-mongodb rsyslog-doc
  rsyslog-openssl | rsyslog-gnutls rsyslog-gssapi rsyslog-relp snort-doc
The following NEW packages will be installed:
  libdaq2 libdumbnet1 libestr0 libfastjson4 libluajit-5.1-2 libluajit-5.1-common
  libnetfilter-queue1 logrotate oinkmaster rsyslog snort snort-common
  snort-common-libraries snort-rules-default
0 upgraded, 14 newly installed, 0 to remove and 0 not upgraded.
Need to get 2931 kB of archives.
After this operation, 12.7 MB of additional disk space will be used.
```

The package manager installs Snort and its supporting rule packages. Snort is an intrusion detection engine that can inspect traffic and write alerts for matching rules.

Check IP address which is 192.168.232.132

```
sang@22080031:~$ sudo ifconfig
[sudo] password for sang:
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.232.132  netmask 255.255.255.0  broadcast 192.168.232.255
        inet6 fe80::20c:29ff:feb9:b76  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:b9:0b:76  txqueuelen 1000  (Ethernet)
        RX packets 187035  bytes 276193146 (276.1 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 40450  bytes 2487527 (2.4 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 190  bytes 18370 (18.3 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 190  bytes 18370 (18.3 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

The server interface output gives 192.168.232.132 on ens33. This is a different lab network from the earlier 192.168.56.x NFS exercise, and the following Snort tests use this address.

Then enter the IP address as a sniffing address.

```
Please use the CIDR form - for example, 192.168.1.0/24 for a block of 256
addresses or 192.168.1.42/32 for just one. Multiple values should be
comma-separated (without spaces).

You can leave this value empty and configure HOME_NET in /etc/snort/snort.conf
instead. This is useful if you are using Snort in a system which frequently
changes network and does not have a static IP address assigned.

Please note that if Snort is configured to use multiple interfaces, it will use
this value as the HOME_NET definition for all of them.

Address range for the local network: 192.168.232.132
```

The installer asks for the local network value used by Snort's default configuration. The screenshot enters the server address; that protects a single host unless the setting is changed to the full subnet.

Task 2: Configure “snort.conf” file

Firstly, backup the configuration file

```
sang@22080031:~$ sudo cp /etc/snort/snort.conf /etc/snort/snort.conf.backup
```

The existing snort.conf is copied to a backup before editing. Keeping the original makes it possible to compare or restore the configuration if a later change fails validation.

```
sang@22080031:~$ sudo nano /etc/snort/snort.conf
```

The main Snort configuration file is opened. This file defines network variables, rule locations, included rules, and inspection behavior.

After that, configure snort with the following code:

```
##################################################
# Step #1: Set the network variables.  For more information, see README.variables
##################################################

# Setup the network addresses you are protecting
#
# Note to Debian users: this value is overriden when starting
# up the Snort daemon through the init.d script by the
# value of DEBIAN_SNORT_HOME_NET s defined in the
# /etc/snort/snort.debian.conf configuration file
#
ipvar HOME_NET 192.168.232.132

# Set up the external network addresses. Leave as "any" in most situations
ipvar EXTERNAL_NET ![192.168.232.132]
# If HOME_NET is defined as something other than "any", alternative, you can
# use this definition if you do not want to detect attacks from your internal

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

```
# Path to your rules files (this can be a relative path)
# Note for Windows users:  You are advised to make this an absolute path,
# such as:  c:\snort\rules
var RULE_PATH /etc/snort/rules
var SO_RULE_PATH /etc/snort/so_rules
var PREPROC_RULE_PATH /etc/snort/preproc_rules

# If you are using reputation preprocessor set these
```

The paired configuration captures set HOME_NET to 192.168.232.132 and EXTERNAL_NET to addresses outside it. With a single-host HOME_NET, rules using these variables focus on traffic to or from this server rather than the entire /24 network. The rule-path settings tell Snort where to find local, shared-object, and preprocessor rules. Correct paths are required for the later include statement to load the custom alerts.

```
# site specific rules
include $RULE_PATH/local.rules
```

The local.rules include makes the five custom rules part of the Snort configuration. Without this line, editing the local rule file would not cause Snort to evaluate those rules.

Command	Description
ipvar HOME_NET 192.168.232.132	Defines the internal (protected) network—Snort considers this your "home" network.
ipvar EXTERNAL_NET ! [192.168.232.132]	Defines the external network—usually all IPs not in the home network (the ! negates).
var RULE_PATH /etc/snort/rules	Sets the path to the directory where Snort rule files are stored.
var SO_RULE_PATH /etc/snort/so_rules	Path for shared object rules (compiled rules).
var PREPROC_RULE_PATH /etc/snort/preproc_rules	Path for preprocessor-specific rules.
Include $RULE_PATH/local.rules	Includes the local custom rules file for this installation of Snort.

The configuration table explains the network and rule-path variables. It connects each setting to the traffic scope or file location used when Snort starts.

Then start, enable and check the service status

```
sang@22080031:~$ sudo systemctl start snort
sang@22080031:~$ sudo sytemctl enable snort
sudo: sytemctl: command not found
sang@22080031:~$ sudo systemctl enable snort
snort.service is not a native service, redirecting to systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable snort
sang@22080031:~$ sudo systemctl status snort
● snort.service - LSB: Lightweight network intrusion detection system
     Loaded: loaded (/etc/init.d/snort; generated)
     Active: active (running) since Tue 2025-04-08 06:04:02 UTC; 4min 32s ago
       Docs: man:systemd-sysv-generator(8)
      Tasks: 2 (limit: 9343)
     Memory: 78.3M
```

The service status shows Snort active after it is started and enabled. The initial systemctl enable attempt failed, but the subsequent command succeeded through the service's SysV integration.

Validates the Snort configuration with:

```
sang@22080031:~$ sudo snort -T -c /etc/snort/snort.conf
Running in Test mode

       --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3702 4343 4848 5250
6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888
8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
PortVar 'SSH_PORTS' defined :  [ 22 ]
PortVar 'FTP_PORTS' defined :  [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined :  [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined :  [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3702 43
43 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 83
00 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined :  [ 2123 2152 3386 ]
Detection:
   Search-Method = AC-Full-Q
    Split Any/Any group = enabled
    Search-Method-Optimizations = enabled
    Maximum pattern length = 20
Tagged Packet Limit: 256
Loading dynamic engine /usr/lib/snort/snort_dynamicengine/libsf_engine.so... done
Loading all dynamic detection libs from /usr/lib/snort/snort_dynamicrules...
WARNING: No dynamic libraries found in directory /usr/lib/snort/snort_dynamicrules.
  Finished Loading all dynamic detection libs from /usr/lib/snort/snort_dynamicrules
Loading all dynamic preprocessor libs from /usr/lib/snort/snort_dynamicpreprocessor/...
  Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor//libsf_sip_preproc.so... done
  Loading dynamic preprocessor library /usr/lib/snort/snort_dynamicpreprocessor//libsf_dns_preproc.so... done
```

The snort -T command loads the configuration in test mode. It checks whether Snort can parse the options and rules without running a live packet test.

```
       --== Initialization Complete ==--

  ,,_     -*> Snort! <*-
 o"  )~   Version 2.9.15.1 GRE (Build 15125)
  ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
          Copyright (C) 2014-2019 Cisco and/or its affiliates. All rights reserved.
          Copyright (C) 1998-2013 Sourcefire, Inc., et al.
          Using libpcap version 1.10.1 (with TPACKET_V3)
          Using PCRE version: 8.39 2016-06-14
          Using ZLIB version: 1.2.11

          Rules Engine: SF_SNORT_DETECTION_ENGINE  Version 3.1  <Build 1>
          Preprocessor Object: SF_REPUTATION  Version 1.1  <Build 1>
          Preprocessor Object: appid  Version 1.1  <Build 5>
          Preprocessor Object: SF_DNP3  Version 1.1  <Build 1>
          Preprocessor Object: SF_FTPTELNET  Version 1.2  <Build 13>
          Preprocessor Object: SF_GTP  Version 1.1  <Build 1>
          Preprocessor Object: SF_SSH  Version 1.1  <Build 3>
          Preprocessor Object: SF_MODBUS  Version 1.1  <Build 1>
          Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
          Preprocessor Object: SF_POP  Version 1.0  <Build 1>
          Preprocessor Object: SF_DCERPC2  Version 1.0  <Build 3>
          Preprocessor Object: SF_SDF  Version 1.1  <Build 1>
          Preprocessor Object: SF_SMTP  Version 1.1  <Build 9>
          Preprocessor Object: SF_SSLPP  Version 1.1  <Build 4>
          Preprocessor Object: SF_DNS  Version 1.1  <Build 4>
          Preprocessor Object: SF_SIP  Version 1.1  <Build 1>

Snort successfully validated the configuration!
Snort exiting
```

The test output ends with a successful validation message. That confirms the configuration can load, but it does not establish that every custom rule will trigger as intended.

```
sang@22080031:~$ sudo nano /etc/snort/rules/local.rules
```

The local.rules file is opened for custom alert definitions. Each rule needs a protocol, traffic direction, match conditions, message, and unique Snort ID.

Now, configure the snort detection rule in the /etc/snort/rules/local.rules with the following code:

```
#Detecting an ICMP Ping Request
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Request"; sid:1000001;)

#Detecting HTTP Traffic on Port 80
alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (msg:"HTTP Traffic"; sid:1000002;)

#Detecting DNS Queries
alert udp $HOME_NET any -> $EXTERNAL_NET 53 (msg:"DNS Query"; sid:1000003;)

#Detecting FTP Login Attempt
alert tcp $HOME_NET 21 -> $EXTERNAL_NET any (msg:"FTP Login Attempt"; sid:1000004;)

#Detecting SSH Login Attempt
alert tcp $HOME_NET 22 -> $EXTERNAL_NET any (msg:"SSH Login Attempt"; sid:1000005;)
```

The five shown rules match ICMP ping, HTTP traffic, DNS queries, and packets involving FTP or SSH ports. The last two are port-based indicators; they do not by themselves prove that a user actually logged in.

Rule explaination:

Rule Type	Protocol	Source → Destination	Ports	Message	SID
ICMP Ping	ICMP	`any` → `$HOME_NET`	`any`	"ICMP Ping Request"	1000001
HTTP Traffic	TCP	`$EXTERNAL_NET` → `$HOME_NET`	any → 80	"HTTP Traffic"	1000002
DNS Query	UDP	`$HOME_NET` → `$EXTERNAL_NET`	any → 53	"DNS Query"	1000003
FTP Login Attempt	TCP	`$HOME_NET` → `$EXTERNAL_NET`	21 → any	"FTP Login Attempt"	1000004
SSH Login Attempt	TCP	`$HOME_NET` → `$EXTERNAL_NET`	22 → any	"SSH Login Attempt"	1000005

The rule table summarizes protocol, direction, port, alert text, and SID for the five custom rules. It is a compact reference for interpreting later alert log entries.

Then restart the service

```
sang@22080031:~$ sudo systemctl restart snort
```

Snort is restarted so it loads the edited local.rules file. A restart is necessary because the running process does not automatically use changes written to disk.

Task 3: Create Five Rules and Test the ICMP Rule

Check client's IP address which is 192.168.232.128

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.232.128  netmask 255.255.255.0  broadcast 192.168.232.25
5
        inet6 fe80::8d95:ea1a:66a9:118b  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:44:24:cc  txqueuelen 1000  (Ethernet)
        RX packets 4446  bytes 2618103 (2.4 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1723  bytes 615020 (600.6 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

The client interface output identifies 192.168.232.128 as the test source. It shares the 192.168.232.0/24 network with the Snort server.

Then ping the server

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ping 192.168.232.132
PING 192.168.232.132 (192.168.232.132) 56(84) bytes of data.
64 bytes from 192.168.232.132: icmp_seq=1 ttl=64 time=0.435 ms
64 bytes from 192.168.232.132: icmp_seq=2 ttl=64 time=1.06 ms
64 bytes from 192.168.232.132: icmp_seq=3 ttl=64 time=0.757 ms
64 bytes from 192.168.232.132: icmp_seq=4 ttl=64 time=0.850 ms
64 bytes from 192.168.232.132: icmp_seq=5 ttl=64 time=0.780 ms
64 bytes from 192.168.232.132: icmp_seq=6 ttl=64 time=0.782 ms
64 bytes from 192.168.232.132: icmp_seq=7 ttl=64 time=6.65 ms
64 bytes from 192.168.232.132: icmp_seq=8 ttl=64 time=0.720 ms
64 bytes from 192.168.232.132: icmp_seq=9 ttl=64 time=0.833 ms
64 bytes from 192.168.232.132: icmp_seq=10 ttl=64 time=0.671 ms
64 bytes from 192.168.232.132: icmp_seq=11 ttl=64 time=0.660 ms
64 bytes from 192.168.232.132: icmp_seq=12 ttl=64 time=0.841 ms
64 bytes from 192.168.232.132: icmp_seq=13 ttl=64 time=0.904 ms
64 bytes from 192.168.232.132: icmp_seq=14 ttl=64 time=0.883 ms
64 bytes from 192.168.232.132: icmp_seq=15 ttl=64 time=0.919 ms
64 bytes from 192.168.232.132: icmp_seq=16 ttl=64 time=0.860 ms
64 bytes from 192.168.232.132: icmp_seq=17 ttl=64 time=0.834 ms
64 bytes from 192.168.232.132: icmp_seq=18 ttl=64 time=0.830 ms
64 bytes from 192.168.232.132: icmp_seq=19 ttl=64 time=0.719 ms
64 bytes from 192.168.232.132: icmp_seq=20 ttl=64 time=0.812 ms
64 bytes from 192.168.232.132: icmp_seq=21 ttl=64 time=0.755 ms
64 bytes from 192.168.232.132: icmp_seq=22 ttl=64 time=0.810 ms
^C
```

The client sends repeated ICMP echo requests to 192.168.232.132 and receives replies. This supplies traffic for the custom ping rule while also showing basic reachability.

Go back to the server and check alert log

```
sang@22080031:~$ sudo cat /var/log/snort/snort.alert.fast
04/08-06:04:39.225306  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan]
[Priority: 3] {UDP} 192.168.232.1:53790 -> 239.255.255.250:1900
04/08-06:04:42.238514  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan]
[Priority: 3] {UDP} 192.168.232.1:53790 -> 239.255.255.250:1900
04/08-06:04:45.254810  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan]
[Priority: 3] {UDP} 192.168.232.1:53790 -> 239.255.255.250:1900
04/08-06:20:45.363893  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan]
[Priority: 3] {UDP} 192.168.232.1:53790 -> 239.255.255.250:1900
04/08-06:20:48.378588  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan]
[Priority: 3] {UDP} 192.168.232.1:53790 -> 239.255.255.250:1900
04/08-06:20:51.379896  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan]
[Priority: 3] {UDP} 192.168.232.1:53790 -> 239.255.255.250:1900
04/08-06:26:30.652364  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2]
UDP} 0.0.0.0:68 -> 255.255.255.255:67
```

The general alert log contains Snort entries, including other scan and traffic alerts. It shows that the daemon is writing events, but those entries need to be matched to a rule ID before interpreting a specific test.

The ping alert are shown with the correct client IP (192.168.232.128)

```
04/08-06:43:35.413868  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:36.446557  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:37.448231  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:38.449984  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:39.451276  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:40.478906  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:41.503097  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:42.527288  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:43.551364  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:44.579535  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:45.581012  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:46.591941  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:43:59.120905  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:00.130777  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:01.132386  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:02.146620  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:03.148317  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:04.150063  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:05.152957  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:06.154051  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:07.171815  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:08.173519  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:09.188223  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:10.212468  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:11.214310  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:12.215832  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
04/08-06:44:13.217705  [**] [1:1000001:0] ICMP Ping Request [**] [Priority: 0] {ICMP} 192.168.232.128 -> 192.168.232.132
```

The final alert capture shows SID 1000001 with source 192.168.232.128 and destination 192.168.232.132. This verifies the custom ICMP rule for the ping test; the document does not show comparable tests for the other four custom rules.